

IPM — PFE Jury Preparation

AI Layer · Semantic Search · Embeddings · Demo Q&A

Elevator pitch (30 seconds)

IPM is an AI-assisted innovation intake platform for DXC. A user submits a business need in natural language; the system classifies it, detects duplicates, semantically searches the DXC catalog, runs gap analysis, scores solutions (IVI), generates delivery recommendations, and exports PDF/DOCX. AI is **hybrid**: embeddings for retrieval, LLM for reasoning, deterministic rules and validation for trust.

Architecture flow

Pitch → [Cache] → [Rules] → [LLM Tags] → [Guards] → Tags

Pitch → [Embed] → Chroma → Duplicate check (≥ 0.80)

Pitch + Tags → [Embed query] → Chroma catalog → [Lexical filter] → Top 4 solutions

Selected solution → [LLM Gap Analysis] → IVI scores → [LLM Recos] → Export

Key numbers (memorize)

Item	Value
Embedding model	BAAI/bge-small-en-v1.5 (384-dim, local)
Vector DB	ChromaDB — cosine / HNSW
Duplicate threshold	≥ 0.80 (max 3 matches)
Catalog search	Pool 40 → lexical filter → return 4
NLP cache	L1: 5 min · L2 Postgres: 24 h
Frontend debounce	900 ms · min pitch: 30 chars
LLM	Groq Llama 3.3 70B or Azure GPT-4o
Chroma collections	business_needs · dxc_catalog

Tech stack one-liner

Next.js 14 · FastAPI · PostgreSQL · ChromaDB · BGE embeddings · Groq/Azure LLM · Langfuse · Docker Compose

Demo pitches to rehearse

- 1. AI HR chatbot — catalog match, no duplicate
- 2. Near-copy of seed BN-2025-001 — duplicate blocked at VC-1
- 3. Data lake + dashboards only — AI domain removed by rules engine

Top 10 jury questions — model answers

1. What problem does IPM solve?

DXC innovation intake is unstructured: business needs arrive as free text and must be qualified, matched to internal catalog solutions, scored, and turned into delivery recommendations. IPM digitizes the full IPM workflow — Sourcing through VC-1 to VC-4 — with AI assisting classification, search, gap analysis, and document generation.

2. Why hybrid AI (rules + LLM + embeddings) instead of just ChatGPT?

Innovation intake needs control and traceability, not only fluency. Rules enforce business logic the LLM can miss (e.g. data dashboard ≠ AI project). Embeddings handle semantic matching at scale. The LLM adds flexible classification and gap reasoning. Post-LLM guards keep outputs on-vocabulary. Langfuse traces decisions. This is cheaper, more auditable, and safer for enterprise than a single black-box prompt.

3. How does semantic catalog search work?

We build a query from pitch + AI tags (objective, domains, impact), embed with BGE using the query prefix, and search the dxc_catalog Chroma collection (cosine similarity). We fetch up to 40 candidates, then apply a lexical overlap filter on features/description to remove semantic false positives. We return top 4 by relevance score (1 – cosine distance). Catalog is seeded from Excel at startup.

4. How does duplicate detection work?

On create, the pitch is embedded as a document and upserted into business_needs in Chroma. We query for similar pitches excluding the new ID. Matches ≥ 0.80 similarity are returned (max 3). VC-1 blocks validation if duplicates exist. Twenty synthetic needs are seeded for demo. This catches paraphrases, not just exact matches.

5. What is the NLP tagging pipeline?

Pitch + horizon → L1 cache (5 min) → L2 Postgres (24 h) → rules engine (KPI/client/AI-vs-Data) → horizon bias in prompt → LLM call → post-parse overrides → validation guards. Tags carry confidence H/M/L. Frontend debounces 900 ms and requires ≥ 30 characters.

6. Is this RAG? Difference between Discovery relevance and IVI?

Not full RAG — we retrieve catalog products but do not inject chunks into a generation prompt. Gap analysis is a separate structured LLM call on the user-selected solution. Discovery relevance = semantic catalog fit (0–1). IVI = multi-dimensional qualification after gap analysis (maturity, expertise, duration, impact, 1–5).

7. How do you reduce hallucinations?

Four layers: pre-LLM rules (hard constraints), structured JSON prompts, post-LLM validation (allow-lists for tags, roles, risks, KPIs), and hybrid scoring (business impact alignment computed deterministically; LLM only phrases catalog-grounded explanations). Gap fit_score is calibrated when missing features exceed matching ones.

8. Why ChromaDB and local BGE instead of pgvector + OpenAI embeddings?

Separation of concerns: Postgres for transactional data, Chroma for vector search with HNSW. Two collections with independent seeding. BGE-small runs locally — no API cost per search. OpenAI embeddings remain configurable via env vars.

9. Main limitations and next steps?

Limits: English-optimized model, no formal precision@k eval, hand-maintained lexical lexicons, no reranker, no auth. Next: evaluation dataset, cross-encoder reranking, multilingual embeddings (bge-m3), auth/audit trail, prompt A/B via Langfuse.

10. Walk us through a live demo.

Happy path: AI HR chatbot pitch → tags → create need → Discovery catalog hits → gap analysis + IVI → Selection → Recos → PDF export. Duplicate: pitch similar to BN-2025-001 → alert at ≥ 0.80 → VC-1 blocked. Rules: data lake + dashboards only → AI domain removed — system does not blindly trust the LLM.

Quick rebuttals

Challenge	Response
Why not fine-tune?	Structured taxonomy + catalog. Rules + prompts give faster, explainable control for PFE scope.
How was 0.80 chosen?	Empirical on demo seeds. Production would tune with labeled duplicate pairs.
Lexical filter seems hacky	Fixes real vector-search false positives. Proper fix = reranker or labeled eval set.
No authentication?	Intentional for PFE demo. Production needs auth, roles, audit trail.